

CS360 – Operating Systems and Architecture I

Homework problems Ch. 3

1. a) Including the initial parent process, how many processes are created by the following program?

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int i;

    for (i = 0; i < 4; i++)
        fork();

    return 0;
}
```

b) Illustrate the process tree with the parent at the root to confirm your answer in question a). Be sure to identify clearly the different children by giving them PIDs of Ch1, Ch2, etc.

2. Some computer systems provide multiple register sets. When a context switch occurs if the new context is already loaded into one of the register sets, the CPU current-register-set pointer is changed to point to the set containing the new context, which takes very little time.
- a) What happens if the new context is in memory rather than in a register set and all the register sets are in use?
- b) Suppose that on a single-register set computer, a context switch takes 10 nanoseconds. Suppose that on a multiple-register set computer, a context switch takes 1 ns if the next process is in one of the existing register sets and 12 ns if the next process is in memory rather than in a register set. What percentage of the time does the next process need to already be in one of the register sets in order to provide a time advantage over the single-register set computer?
3. Describe the actions taken by a kernel to context-switch between processes.

4. Using the program shown below, explain what the output will be at lines X and Y.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

#define SIZE 5

int nums[SIZE] = {0,1,2,3,4};

int main()
{
    int i;
    pid_t pid;

    pid = fork();

    if (pid == 0) {
        for (i = 0; i < SIZE; i++) {
            execlp("/bin/ls", "ls", NULL);
            printf("CHILD: %d\n", nums[i]);          /* LINE X */
        }
    }
    else if (pid > 0) {
        wait(NULL);
        for (i = 0; i < SIZE; i++)
            printf("PARENT: %d\n ", nums[i]);      /* LINE Y */
    }

    return 0;
}
```

5. What are the benefits and the disadvantages of each of the following? Consider both the system level and the programmer level.
- Synchronous and asynchronous communication
 - Automatic and explicit buffering
 - Send by copy and send by reference
 - Fixed-sized and variable-sized messages
6. Give an example of a situation in which ordinary pipes are more suitable than named pipes and an example of a situation in which named pipes are more suitable than ordinary pipes.

7. Given the following program and two different sets of output generated from the same program, draw the process tree for each output and explain what is happening.

Program: Create_process.c

```
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t ch1, ch2, ch3;

    printf("1: %d\n", getpid());

    /* fork a child process */
    ch1 = fork();

    /* fork another child process */
    ch2 = fork();

    /* and fork another */
    ch3 = fork();

    printf("2: %d from %d\n", getpid(), getppid());

    return 0;
}
```

Output set 1:

```
eins@csa2:~/cs360/examples/c_examples$ ./create_process
1: 21273
2: 21273 from 7320
2: 21276 from 21273
2: 21274 from 21273
2: 21275 from 21273
2: 21279 from 21275
2: 21278 from 1
2: 21280 from 21277
2: 21277 from 1
eins@csa2:~/cs360/examples/c_examples$
```

Output set 2:

```
eins@csa2:~/cs360/examples/c_examples$ ./create_process
1: 22551
2: 22551 from 7320
2: 22553 from 1
2: 22554 from 22552
eins@csa2:~/cs360/examples/c_examples$ 2: 22552 from 1
2: 22555 from 1
2: 22556 from 1
2: 22558 from 1
2: 22557 from 1
```